Get unlimited access to the best of Medium for less than $1/week. Become a member

Practice SQL JOINS with me: A Hands-on Tutorial

Case Study: Music Streaming Service

Richard Warepam · Follow
Published in ILLUMINATION'S MIRROR · 9 min read · Feb 22, 2024

136

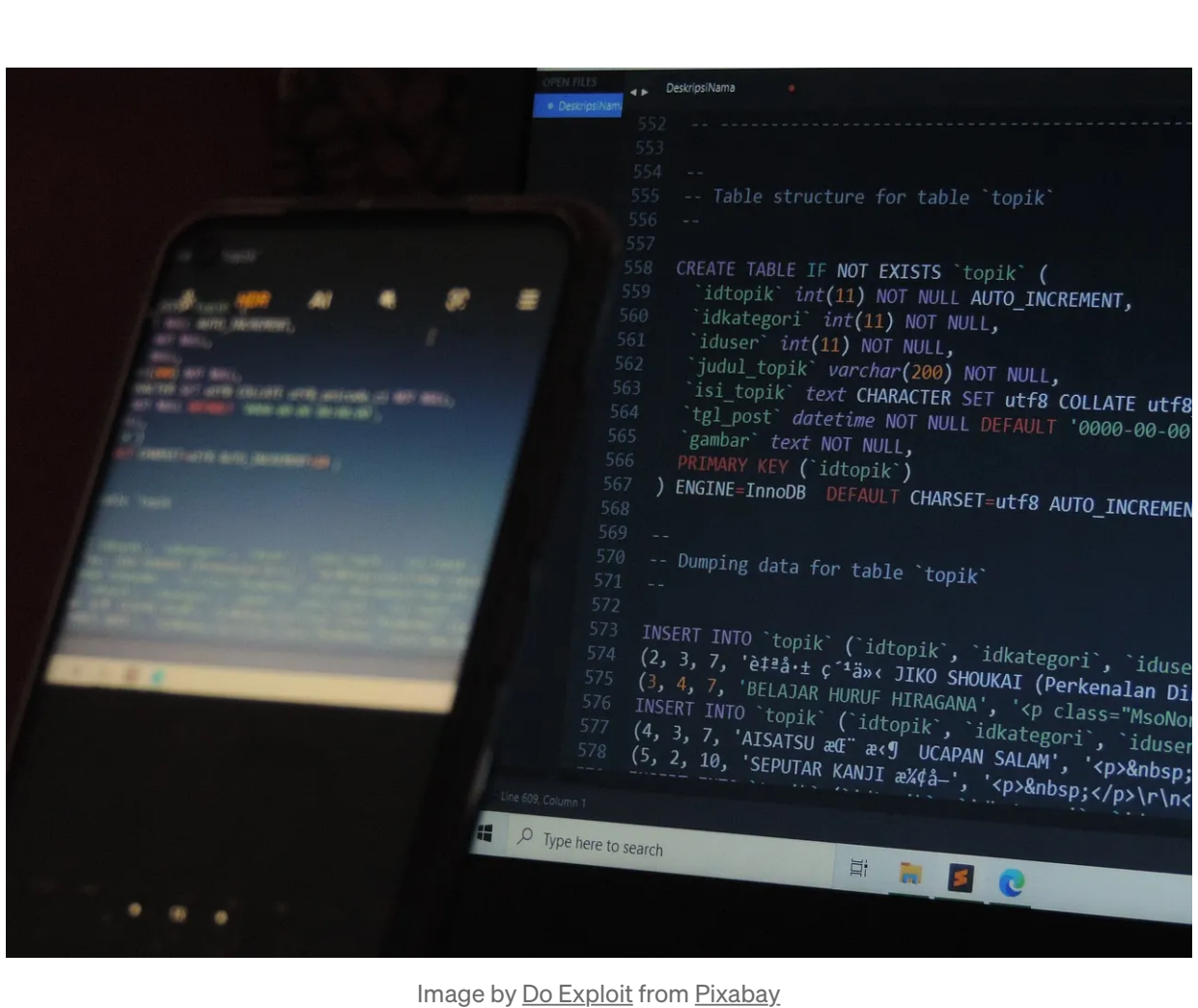


Image by Do Exploit from Pixabay

Hey there! In this article, we're not diving into SQL joins.

However, if that's what you're keen on mastering, I've got just the thing for you. Check out "*The Ultimate Guide to Mastering JOINS in SQL*."

The Ultimate Guide to Mastering "JOINS" in SQL
medium.com

It's a hit, with over 400 people already soaking up its wisdom.

Don't miss out; give it a read, and then swing back here to get your hands dirty with some practical exercises.

So, back to our main story. We're about to roll up our sleeves and tackle some queries together.

I'll walk you through a case study, leading the charge on the first 7 questions. The final 3? They're your chance to shine; tackle them on your own.

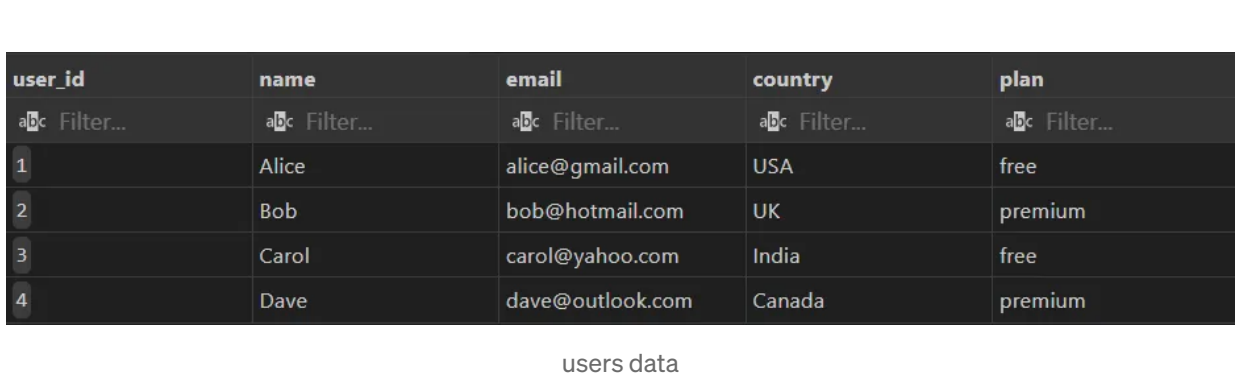
. . .

Imagine we're working with a music streaming service's database, packed with these tables:

users:

Here, we capture the essence of our users who've signed up. This table is decked out with columns like:

- **user_id**: The unique identifier of the user
- **name**: The name of the user
- **email**: The email address of the user
- **country**: The country where the user is located
- **plan**: The subscription plan of the user, either 'free' or 'premium'



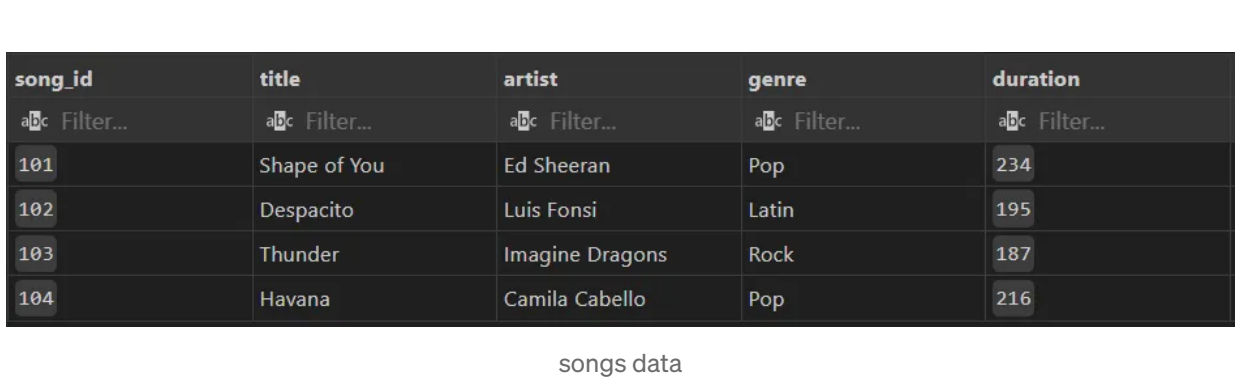
user_id	name	email	country	plan
1	Alice	alice@gmail.com	USA	free
2	Bob	bob@hotmail.com	UK	premium
3	Carol	carol@yahoo.com	India	free
4	Dave	dave@outlook.com	Canada	premium

users data

songs:

This table hits the right notes with details on the service's song library. It includes:

- **song_id**: The unique identifier of the song
- **title**: The title of the song
- **artist**: The name of the artist who performed the song
- **genre**: The genre of the song
- **duration**: The duration of the song in seconds



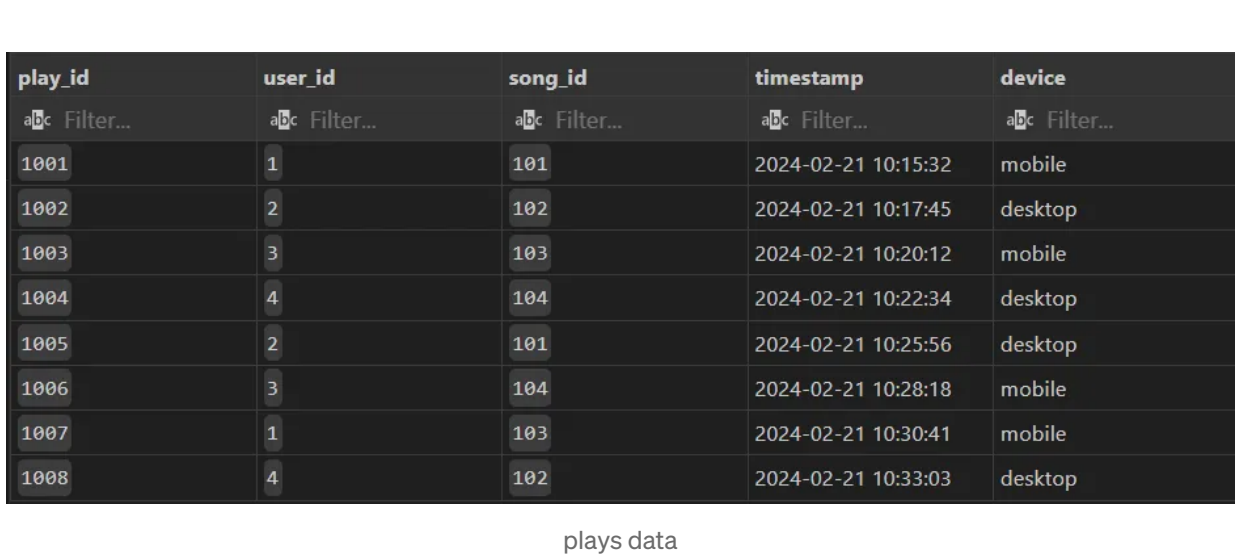
song_id	title	artist	genre	duration
101	Shape of You	Ed Sheeran	Pop	234
102	Despacito	Luis Fonsi	Latin	195
103	Thunder	Imagine Dragons	Rock	187
104	Havana	Camila Cabello	Pop	216

songs data

plays:

And here, we track every time a user plays a song. It's filled with information on:

- **play_id**: The unique identifier of the play
- **user_id**: The user who played the song
- **song_id**: The song that was played
- **timestamp:** The date and time when the song was played
- **device:** The device used to play the song, either 'mobile' or 'desktop'



play_id	user_id	song_id	timestamp	device
1001	1	101	2024-02-21 10:15:32	mobile
1002	2	102	2024-02-21 10:17:45	desktop
1003	3	103	2024-02-21 10:20:12	mobile
1004	4	104	2024-02-21 10:22:34	desktop
1005	2	101	2024-02-21 10:25:56	desktop
1006	3	104	2024-02-21 10:28:18	mobile
1007	1	103	2024-02-21 10:30:41	mobile
1008	4	102	2024-02-21 10:33:03	desktop

plays data

. . .

🔌 Plug-in

Before we continue with this article, ***make sure to elevate your data skills with my expert eBooks—the culmination of my experiences and insights.***

Support my work and enhance your journey. Check them out:

- eBook 1: *Personal INTERVIEW Ready "SQL" CheatSheet*
- eBook 2: *Personal INTERVIEW Ready "Statistics" Cornell Notes*
- Data Science Bundle (**Cheapest**): *The Ultimate Data Science Bundle: Complete*
- Interview Ready Bundle (**Cheapest**): *Data Science Interview Ready Bundle*

Best Selling eBook: *Top 50+ ChatGPT Personas for Custom Instructions*

. . .

Questions and Answers

Now, let's see how we can use SQL joins to answer some questions based on the data from the case study.

Q1: How many songs did each user play?

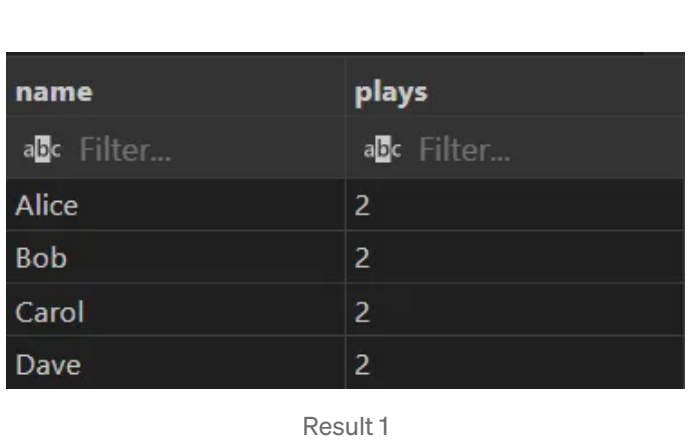
We're going to look at the number of times users have hit play.

To do this, we need to bring the '*users*' and '*plays*' tables together, like linking hands, where '*user_id*' is our handshake.

- We'll use an **INNER JOIN** for this task.
- Then, by grouping our users with the '**GROUP BY**' clause, we can keep everything neat and tidy.
- Lastly, we'll call on the **COUNT** function to reveal the total plays for each user.

Here is the SQL query:

```sql
SELECT u.name, COUNT(p.play_id) AS plays
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
GROUP BY u.name;
```



name	plays
Alice	2
Bob	2
Carol	2
Dave	2

Result 1

Q2: What are the titles and genres of the songs played by Alice?

Alright, let's tackle this question: we need to find the titles and genres of songs that Alice jammed to.

Here's how we're going to break it down: we'll be diving into the '*users*', '*plays*', and '*songs*' tables, connecting them with '*user_id*' and '*song_id*' as our key pieces.

- We'll be using an **INNER JOIN.**

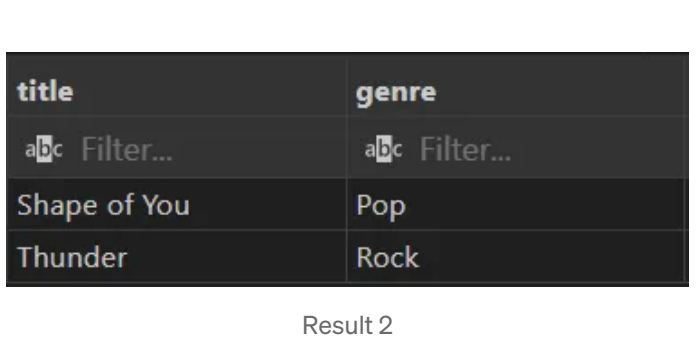
This is just our cool way of making sure we're looking at data that matches up perfectly across all tables, so we catch every tune Alice played.

- Then, to zero in on Alice's music choices, we'll use the **WHERE** clause.

This lets us filter through the data to find just the songs she enjoyed.

Here is the SQL query:

```sql
SELECT s.title, s.genre
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
INNER JOIN music.songs s
ON p.song_id = s.song_id
WHERE u.name = 'Alice';
```



title	genre
Shape of You	Pop
Thunder	Rock

Result 2

Q3: Which users have played songs by Ed Sheeran?

Now, Let's figure out which users have been tuning into Ed Sheeran's hits.

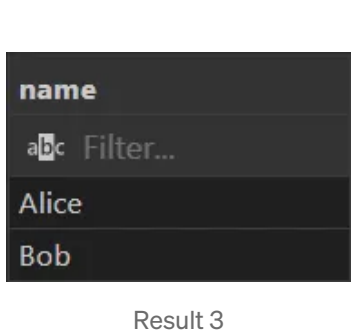
To solve this, we're going to sift through the '*users*', '*plays*', and '*songs*' tables, linking them up using '*user_id*' and '*song_id*'.

- We'll keep using an **INNER JOIN**, our go-to move for making sure we only spotlight rows where there's a match across the board.
- Then, to find Ed Sheeran's audience, we'll dial in with the **WHERE** clause, focusing squarely on his tracks.

This strategy will guide us straight to a list of users who can't get enough of his music.

Here is the SQL query:

```sql
SELECT u.name
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
INNER JOIN music.songs s
ON p.song_id = s.song_id
WHERE s.artist = 'Ed Sheeran';
```



name
Alice
Bob

Result 3

Q4: Which songs have not been played by any users?

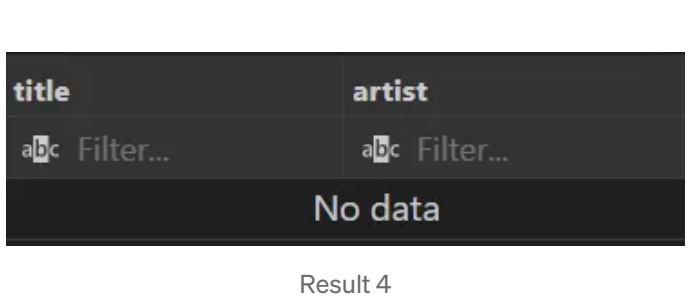
Let's dive into discovering which tracks haven't yet had the chance to grace anyone's playlist.

We're going to explore the '*songs*' and '*plays*' tables, connecting them through the '*song_id*'.

- For this task, we'll use a **LEFT JOIN**.
- If a song doesn't match with any plays, it remains empty, indicated by **NULL** in the '*plays*' table columns related to it.
- To highlight these yet-to-be-played tracks, we'll employ the **WHERE** clause, looking specifically for those instances where '*play_id*' is a telling **NULL**.

Here is the SQL query:

```sql
SELECT s.title, s.artist
FROM music.songs s
LEFT JOIN music.plays p
ON s.song_id = p.song_id
WHERE p.play_id IS NULL;
```



title	artist
No data	

Result 4

Q5: Which users have a premium plan and have played songs on a mobile device?

To get this task started, we're going to merge information from the '*users*' and '*plays*' tables, connecting them with the '*user_id*' as our linking point.

- For this journey, an **INNER JOIN** is our ticket.

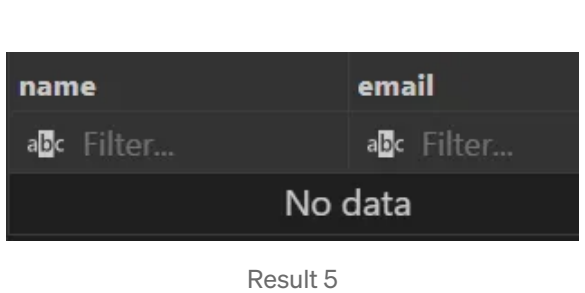
It helps us find those exact matches where the details in both tables sync up perfectly.

- But here's where it gets interesting: we'll use the **WHERE** clause to fine-tune our search.

We're looking for the elite group of users who have premium access and prefer their beats on the go, using their mobiles.

Here is the SQL query:

```sql
SELECT u.name, u.email
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
WHERE u.plan = 'premium'
AND p.device = 'mobile';
```



name	email
No data	

Result 5

Q6: Which songs have been played by both Alice and Bob?

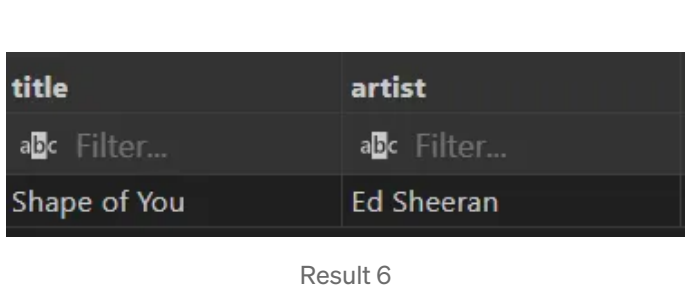
So, identifying the songs that both Alice and Bob have rocked out to? — We're going to sift through the '*users*', '*plays*', and '*songs*' tables, using '*user_id*' and '*song_id*' as our guides to connect the dots.

- We'll stick with an **INNER JOIN** for this task.
- But that's just the start. To pinpoint our musical duo, Alice and Bob, we'll bring in the **WHERE** clause, filtering the data to focus exclusively on them.
- And here's where the magic happens: we'll use the **INTERSECT** operator.

This tool helps us uncover the songs that both Alice and Bob have enjoyed, showing us where their musical tastes intersect.

Here is the SQL query:

```sql
SELECT s.title, s.artist
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
INNER JOIN music.songs s
ON p.song_id = s.song_id
WHERE u.name = 'Alice'
INTERSECT
SELECT s.title, s.artist
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
INNER JOIN music.songs s
ON p.song_id = s.song_id
WHERE u.name = 'Bob';
```



title	artist
Shape of You	Ed Sheeran

Result 6

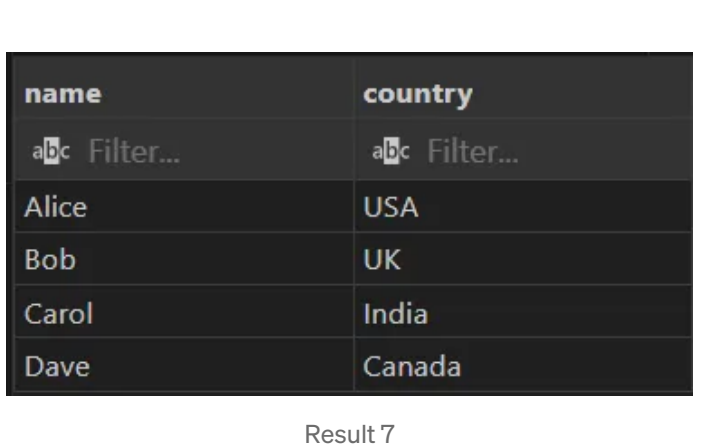
Q7: Which users have played songs from different genres?

We'll navigate this by linking the '*users*', '*plays*', and '*songs*' tables, with '*user_id*' and '*song_id*' serving as our trusted guides.

- An **INNER JOIN** is our tool of choice here, ensuring we only consider rows where the connections are clear and consistent across all tables involved.
- Now, to get a real sense of each user's musical adventure, we'll organize our data with the **GROUP BY** clause, clustering the information by user.
- But we're not stopping there. To identify those adventurers who've ventured through various musical landscapes, we'll use the **HAVING** clause.

Here is the SQL query:

```sql
SELECT u.name, u.country
FROM music.users u
INNER JOIN music.plays p
ON u.user_id = p.user_id
INNER JOIN music.songs s
ON p.song_id = s.song_id
GROUP BY u.name, u.country
HAVING COUNT(DISTINCT s.genre) > 1;
```



name	country
Alice	USA
Bob	UK
Carol	India
Dave	Canada

Result 7

. . .

Questions for You

Now that you've gotten the hang of SQL joins, it's your turn to put those skills to the test.

I've set up 3 questions for you to tackle using the tables and data from our case study. Here's your chance to show off what you've learned!

Q8: Can you figure out how many songs from each genre have been enjoyed by users?

> ***Here's a hint to get you started:*** *Dive into the 'songs' and 'plays' tables and link them using the 'song_id'. Then, group the results by genre with the GROUP BY clause and count them up with the COUNT function. This will give you a clear picture of the musical diversity our users are exploring.*

Q9: Who's been listening to tunes that stretch beyond 200 seconds?

> ***Here's how you can find out:*** *Bring together the 'users', 'plays', and 'songs' tables by joining them on 'user_id' and 'song_id'. Use the WHERE clause to sift through the tracks based on their 'duration'. This will help you identify the users who have a taste for longer jams.*

Q10: Which tracks have made the rounds among users from the same country as the artist?

> *For this one, you'll need to use the 'users', 'plays', and 'songs' tables again, connecting them through 'user_id' and 'song_id'. Apply the WHERE clause to filter your search to match the 'country' of the user with the 'artist'. It's a unique way to see how local flavors resonate with listeners.*

Ready to dive in? These questions are your playground to apply what you've learned and deepen your understanding of SQL joins.

Share your answers and thoughts in the comments, and let's learn from each other!

. . .

Go ahead and drop your answers right in the comments below.

This isn't just a test of what you know; it's a chance to really think deeply about how JOINs work in the wild, solving actual data puzzles.

Diving into these questions is like taking another step toward SQL wizardry. You're arming yourself with the tools to navigate complex data relationships and pull off some pretty slick data retrieval moves.

So, let's see what you've got! Share your insights, and let's learn from each other's journeys into the heart of SQL mastery.

. . .

> *Hey, If you need any Data Science Project related gigs and Ghostwriting gigs:*
>
> 📥 ***Contact:*** *richardwarepam16@gmail.com*
>
> 💡 *Join my community of learners!* ***Subscribe now*** *and instantly receive 2 FREE eBooks:* ***"1000 Money Making ChatGPT Prompts"*** *and* ***"Full Stack Data Science Project Prompts."*** *— https://yourdataguide.substack.com/*
>
> ⭐ ***Visit My Gumroad Shop***
>
> **Note:** *This article contains some plugins for my eBooks. If you like my content, please make sure to support me and appreciate my work.*
> *As MPP is not yet available in India, I make some cash through these.*

Sql · Sql Joins · Data Science · Data Manipulation · Data Analysis

136

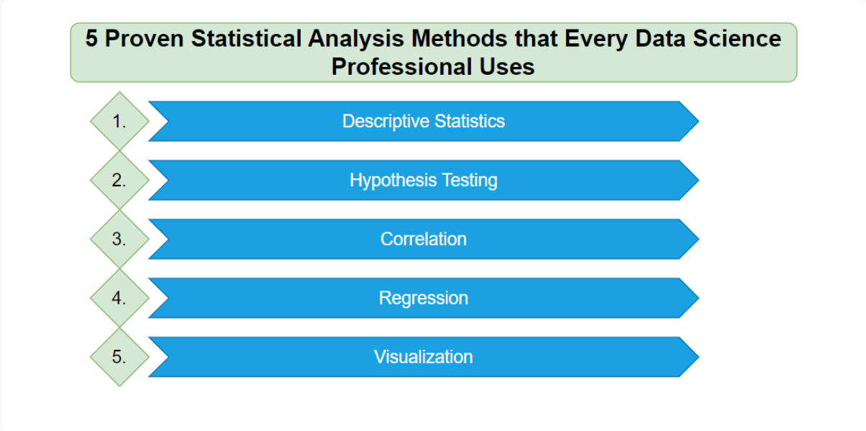
Written by Richard Warepam

Follow

6.2K Followers · Writer for ILLUMINATION'S MIRROR

Data Scientist & Writer | Google Certified Data Analyst | As a Mentor - Writes on Data Science and AI | My eBooks: https://codewarepam.gumroad.com/

More from Richard Warepam and ILLUMINATION'S MIRROR


5 Proven Statistical Analysis Methods that Every Data Science Professional Uses
1. Descriptive Statistics
2. Hypothesis Testing
3. Correlation
4. Regression
5. Visualization

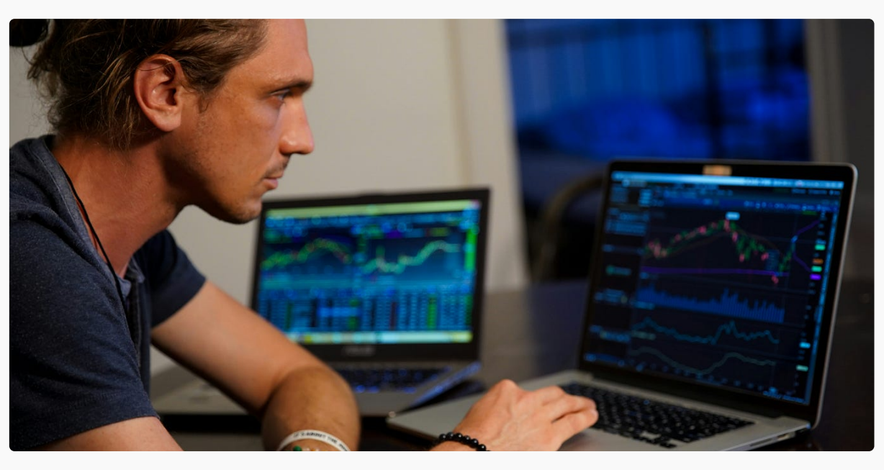
Richard Warepam in ILLUMINATION

How To Perform Statistical Analysis Using Python: The...

5 Proven Methods that Every Data Science Professional Uses

8 min read · Mar 5, 2024

556 2



Sohan Dillikar in ILLUMINATION'S MIRROR

Here's How I Make $50/Day By Only Placing One Trade

Here is my super simple one-trade-a-day strategy that you can use to consistently...

✦ · 5 min read · Mar 8, 2024

1.5K 27

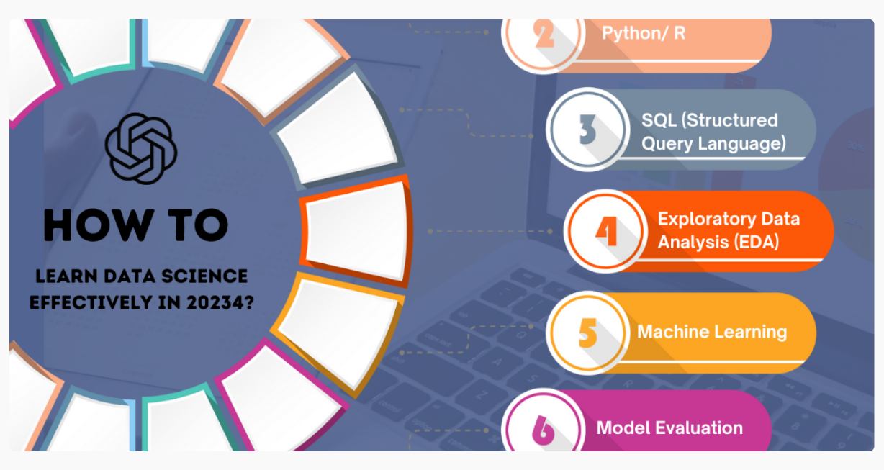
Evan Crosby in ILLUMINATION'S MIRROR

5 Websites Where Rich People Literally Give Away Free Money

One way to make money online is to ask generous wealthy people to give you free...

✦ · 2 min read · Jun 16, 2023

2.7K 44


HOW TO
LEARN DATA SCIENCE EFFECTIVELY IN 20234?
2 Python/ R
3 SQL (Structured Query Language)
4 Exploratory Data Analysis (EDA)
5 Machine Learning
6 Model Evaluation

Richard Warepam in ILLUMINATION

How to Effectively Learn Data Science in 2024

Self-Study Guide to Get Ready for Data Science Jobs

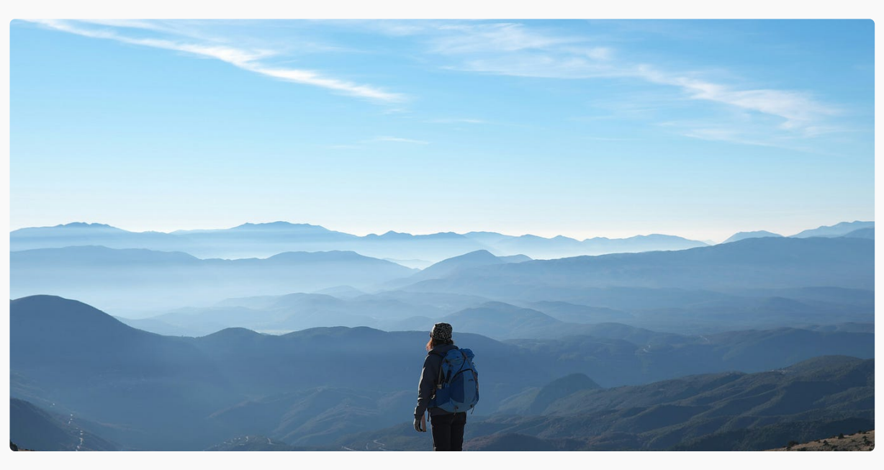
9 min read · Dec 30, 2023

1.3K 8

See all from Richard Warepam

See all from ILLUMINATION'S MIRROR

Recommended from Medium



Sze Zhong LIM in Data And Beyond

Mastering Exploratory Data Analysis (EDA): Everything You...

A systematic approach to EDA your data and prep it for machine learning.

18 min read · Apr 6, 2024

327 3

Modular
M
Mojo
One language for all AI programming
Fabian Tschopp - talk at SINTEF - 17th of January 2024

Dylan Cooper in Stackademic

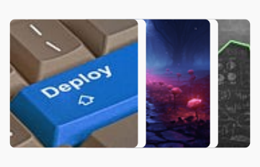
Mojo, 90,000 Times Faster Than Python, Finally Open Sourced!

On March 29, 2024, Modular Inc. announced the open sourcing of the core components o...

✦ · 10 min read · Apr 8, 2024

2.6K 20

Lists



Predictive Modeling w/ Python

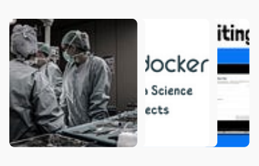
20 stories · 1105 saves

Practical Guides to Machine Learning

10 stories · 1319 saves

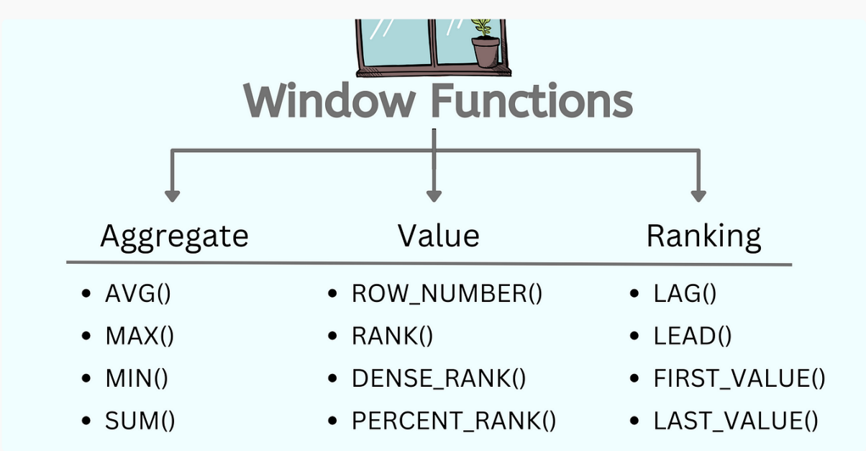
ChatGPT prompts

47 stories · 1432 saves



Coding & Development

11 stories · 565 saves



Hasan Hüseyin Coşgun

Learn SQL Window Functions With Queries

Time to discover the world of Window Functions with examples; lead( ), lag( ),...

7 min read · Nov 2, 2023

87

Hazel Paradise

How I Create Passive Income With No Money

many ways to start a passive income today

5 min read · Mar 27, 2024

5.7K 136

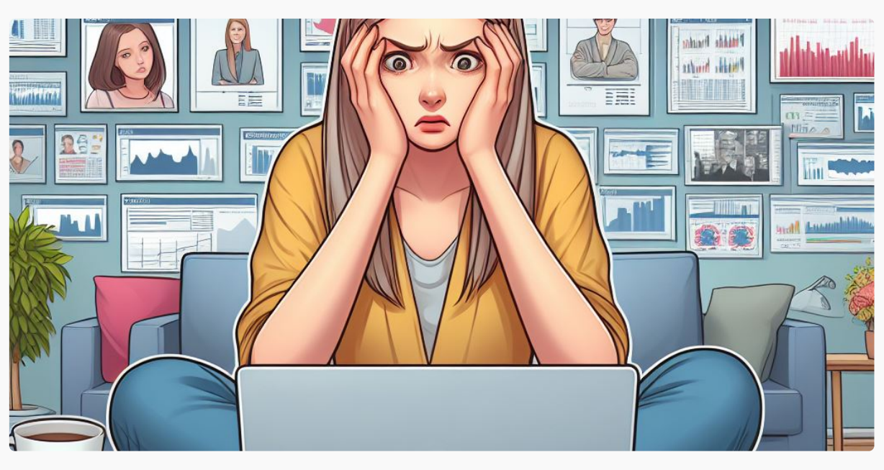
Abrar Muhtasim in DataDrivenInvestor

A Python Data Analysis Project to Understand Hotel Cancellations

Data Cleaning, Analysis, Visualization, Feature Selection, Predictive Modeling

14 min read · Nov 13, 2023

101



Luna (Van) Doan

Data Portfolio That Stands Out in the Crowd: Framework, Resource...

Ditch Kaggle and use this instead!

6 min read · Mar 19, 2024

228 6

See more recommendations